KGiSL MicroCollege
Genius as a Service (GaaS)
TECHNICAL TEAM

KGCAS
co-Kreate your Genius

KG COLLEGE OF ARTS AND SCIENCE
Autonomous Institution | Affiliated to Bharathiar University
Accredited with A++ Grade by NAAC
ISO 9001:2015 Certified Institution
KGiSL Campus, Saravanampatti, Coimbatore – 641 035

C PROJECT REPORT ON

SIMPLE CONSOLE CALCULATOR

SUBMITTED TO

The Technical Department

KG College of Arts and Science

SUBMITTED BY

Student Name	Roll No.	Course & Batch
AADHARSHA	25BCS101	B.Sc CS
AJAYKUMAR	25BCS102	B.Sc CS
AMALIPRABHA	25BCS103	B.Sc CS
ARSHAD	25BCS104	B.Sc CS
ASHWANTH	25BCS105	B.Sc CS

Academic Year: **2025 – 2026**

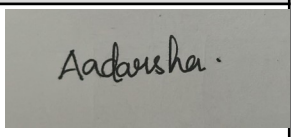
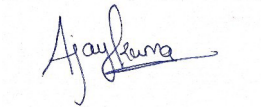
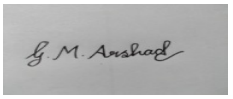
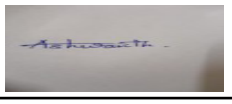
Semester: **I**

DECLARATION

We hereby declare that the project entitled "**SIMPLE CONSOLE CALCULATOR**" submitted in partial fulfilment of the requirements for the course **F.Com** is a **record of our original work** carried out under the guidance of , Technical Trainer, Technical Department, KGiSL MicroCollege.

We further declare that this project has not been submitted previously, either in part or in full, for the award of any degree, diploma, or other similar title, at this or any other institution.

We have acknowledged all sources of information used in this project wherever applicable.

Student Name	Roll No.	Course & Batch	Signature
AADHARSHA	25BCS101	BSC.CS–2025	
AJAYKUMAR	25BCS102	BSC.CS–2025	
AMALIPRABHA	25BCS103	BSC.CS–2025	
ARSHAD	25BCS104	BSC.CS–2025	
ASHWANTH	25BCS105	BSC.CS–2025	

Place: COIMBATORE

Date: 17/03/2026

CERTIFICATE

This is to certify that the mini project report entitled "**SIMPLE CONSOLE CALCULATOR"** has been successfully completed and submitted by the students of **1 BSC. CS 'B', in partial fulfillment of the requirements for the award of the degree/diploma during the academic year **2025-2026**.

It is further certified that this project is the **original work** of the students and has not been previously submitted for the award of any degree, diploma, or certificate at this or any other institution.

Project Team:

S. No.	Name	Roll Number
1.	AADHARSHA	25BCS101
2.	AJAYKUMAR	25BCS102
3.	AMALIPRABHA	25BCS103
4.	ARSHAD	25BCS104
5.	ASHWANTH	25BCS105

Guide: **Head of the Department**

(Technical Trainer, KGCAS)

Mr.MOSES JOHN PRAKASH

(Technical Head, KGCAS)

Mr.BOOPALAN

_____ _____

Place: COIMBATORE

Date:17/03/2026

ABSTRACT

The project **"Simple Console Calculator"** is developed using the C programming language to perform basic arithmetic operations in an efficient and user-friendly manner. The main objective of this project is to provide a simple, reliable, and interactive tool that allows users to carry out everyday mathematical calculations directly through the console.

The system enables users to perform fundamental operations such as **addition, subtraction, multiplication, division, and modulus**. The user is prompted to enter two numerical values and choose an operation from a predefined menu. The calculator processes the input, performs the selected operation using appropriate control structures, and displays the result instantly. It also includes basic error-handling features, such as checking for division by zero, ensuring accuracy and preventing invalid operations.

This project demonstrates the practical application of **functions, conditional statements, loops, and user input handling** in C programming. The modular structure of the program enhances readability, maintainability, and reusability of code, making it suitable for beginners who want to strengthen their programming fundamentals.

Through the development of this console-based calculator, essential concepts such as **structured programming, logical problem solving, algorithmic thinking, and clean code practices** have been effectively implemented. This project not only serves as a learning tool but also highlights how simple C programs can form the foundation of more advanced computational applications in real-world software development.

Through this project, practical concepts of **structured programming, control flow, modular design, and data processing** have been implemented successfully. It not only enhances programming logic and problem-solving skills but also bridges the gap between academic learning and real-world application in the field of software development.

TABLE OF CONTENTS

S. No.	Topics	Page. No.
1	**Introduction**	
1.1	Objective	1
1.2	Overview	1
1.3	Features	2
2	**Program Analysis**	
2.1	Problem Definition	3
2.2	Project Overview	3
2.3	Scopes and Limitations	4
3	**System Design**	
3.1	Modules	5
4	**Methodology**	
4.1	Problem Analysis	5
4.2	System Design	6
4.3	Program Development	6
4.4	Testing and Validation	7
4.5	Result and Documentation	7
4.6	Algorithm	8
4.7	Flowchart	9
5	**Source Code**	11

6	**Input and Output Screens**	17

7	**Discussion and Conclusion**	
7.1	Discussion	18
7.2	Conclusion	19
8	**References**	19

1. INTRODUCTION

1.1 Objective

- **To design and develop a Simple Console Calculator using the C programming language.**
- **To perform basic arithmetic operations such as addition, subtraction, multiplication, division, and modulus through an interactive console interface.**
- **To demonstrate the use of fundamental programming concepts — variables, data types, operators, conditional statements, loops, and functions.**
- **To enable users to enter numerical inputs and select operations in a clear and user-friendly manner.**
- **To implement error-handling techniques such as checking for invalid operators and division by zero.**
- **To enhance understanding of structured programming and improve logical and analytical problem-solving skills.**

1.2 Overview

Perform basic arithmetic operations such as addition, subtraction, multiplication, division, and modulus.

- **Enter two numerical values and choose the desired mathematical operation.**
- **View instant results calculated automatically based on the selected operation.**
- **Handle invalid inputs, such as unsupported operators or division by zero, ensuring safe and error-free calculations.**
-

1.3 Features

- **Interactive menu-driven interface for selecting arithmetic operations.**
- **Uses variables and operators to perform all basic calculations.**
- **Supports addition, subtraction, multiplication, division, and modulus operations.**
- **Allows users to enter multiple sets of numbers for repeated calculations.**
- **Includes error-handling for invalid inputs and division by zero.**
- **Displays the final calculated result clearly before returning to the main menu.**

2. PROGRAM ANALYSIS

2.1 Problem Definition

A basic computing tool is required to perform quick arithmetic calculations, including:

- **Accepting two numerical inputs from the user.**
- **Allowing the user to choose an arithmetic operation such as addition, subtraction, multiplication, division, or modulus.**
- **Processing the calculation accurately and efficiently.**
- **Displaying the final result clearly to the user for further use.**

2.2 Project Overview

Aspect	Description
Language Used	C
Compiler	GCC / Dev-C++ / Turbo C++
Concepts Used	Loops, Functions, Arrays, Conditional StatementsVariables, Operators, Conditional Statements, Loops, Functions
I/O Type	Console-based
Optional	Error Handling (e.g., division by zero)

2.3 Scopes and Limitations

Scope:

- Suitable for students, beginners, and basic mathematical operations.
- Useful for quick calculations in a console environment.
- Can be easily extended to scientific calculator features such as square root, power, trigonometric functions, etc.

Limitations:

- Works only in text (console) mode — no graphical interface.
- Performs only basic operations (addition, subtraction, multiplication, division, modulus).
- Does not support complex expressions (e.g., 5 + 7 × 3) in a single input.
- User must enter numbers and operators manually each time.

3. SYSTEM DESIGN

3.1 Modules

The **Simple Console Calculator** is divided into several functional modules, where each module handles a specific operation of the program. These modules work together to perform calculations efficiently and provide a clear, user-friendly experience for anyone needing quick arithmetic results.

Module Name	Description
Menu Display Module	This module displays the list of available arithmetic operations such as addition, subtraction, multiplication, division, and modulus. It serves as the main interface where the user chooses which calculation to perform.
Input Module	This module accepts numerical inputs from the user. It prompts the user to enter two numbers and validates the input to ensure only valid numeric values are processed.
Operation Processing Module	This module performs the selected arithmetic operation based on user input. It applies the correct mathematical formula for addition, subtraction, multiplication, division, or modulus. It

ModuleName	Description
Result Display Module	output in a simple and readable way to help the user understand the calculation outcome.
Repetition / Loop Module	This module allows the user to perform multiple calculations without restarting the program. After each operation, it prompts the user to continue or exit.
Exit/Termination Module	This module safely ends the program when the user chooses to exit. It displays a closing message (e.g., “Thank you for using the calculator!”) to indicate successful program termination.
also handles special cases such as division by zero.	
This module shows the final computed result clearly on the console. It formats the	

Module Interaction Overview

1. **Menu Display** → shows the list of available arithmetic operations (Add, Subtract, Multiply, Divide, Modulus).
2. **Input Module** → accepts two numbers from the user and validates the inputs.
3. **Operation Processing** → performs the selected calculation and handles errors (e.g., division by zero).
4. **Result Display** → presents the computed result clearly on the screen.
5. **Repetition Module** → asks the user whether they want to perform another calculation.
6. **Exit Module** → terminates the program gracefully with a thank-you message.

4.METHODOLOGY

The development of the Simple Console Calculator project follows a structured methodology consisting of five major stages: Problem Analysis, System Design, Program Development, Testing & Validation, and Result & Documentation.

4.1. Problem Analysis

In this stage, the need for a basic and reliable calculator system was studied and analyzed.

The main problems identified were:

- Manual calculations can lead to errors.

- Traditional calculators may not be always available.
- Beginners need a simple tool to perform arithmetic operations for learning and practice.

Based on these observations, clear objectives were defined — such as accepting user inputs, performing arithmetic operations, displaying results accurately, and enabling repeated calculations.

The overall problem was broken down into smaller tasks to make the program modular, simple, and efficient.

4.2. System Design

During the design phase, the logical structure of the calculator was prepared.

The main components identified were:

- Menu Display Module
- Input Module
- Operation Processing Module
- Result Display Module
- Repetition / Loop Module
- Exit Module

Data structures were minimized because the system handles simple numeric values. Variables were identified for operands, operator selection, and result storage.

Flowcharts and algorithms were prepared to illustrate the flow of the program and the sequence of operations before actual coding.

4.3. Program Development

This phase focuses on implementing the design into actual C code.

The program was written using structured programming concepts, including functions to handle:

- Displaying the menu

- Taking user inputs
- Performing arithmetic operations
- Displaying results
- Handling division-by-zero conditions
- Repeating or terminating the program based on user choice

Control structures like if-else, switch-case, and loops were used to make the system interactive and fully functional.

4.4. Testing and Validation

After coding, each module was tested individually for correctness.

Test cases were prepared for different scenarios, such as:

- Valid numeric inputs
- Invalid operators
- Division by zero
- Repeated calculations
- Boundary values (e.g., zero, negative numbers)

The accuracy of operations (addition, subtraction, multiplication, division, modulus) was verified manually to ensure the results were correct.

The system successfully passed all test cases and was validated as an error-free and reliable calculator.

4.5. Result and Documentation

The final program provided the expected output — performing arithmetic operations accurately and displaying the results clearly.

A menu-driven interface allowed repeated calculations without restarting the program.

Proper documentation was prepared including the:

- Algorithm
- Flowchart
- Module descriptions
- Sample I/O screens
- Conclusion

This ensures that the project is easy to understand, maintain, and enhance.

4.6. Algorithm

An algorithm is a step-by-step procedure that defines how a problem is solved logically.

In the Simple Console Calculator, the algorithm plays a crucial role by defining how the program accepts two numbers, identifies the operation to be performed, calculates the result, and displays the output.

Purpose of the Algorithm

- To define the logical flow of arithmetic operations.
- To ensure accuracy in calculations.
- To provide a clear structure before coding.
- To simplify debugging, understanding, and future improvements.

Algorithm for the Simple Console Calculator

1. Start the program and display the calculator title.
2. Display the menu showing available operations (Add, Subtract, Multiply, Divide, Modulus).
3. Accept the user's choice of operation.
4. Accept two numbers from the user.
5. Validate the inputs.
6. Based on the selected operation:

    - Perform addition
    - Or subtraction
    - Or multiplication
    - Or division (check division by zero)
    - Or modulus
7. Display the result of the calculation.
8. Ask the user whether they want to perform another calculation.
9. If yes, go back to the menu; otherwise terminate the program.
10. Display a thank-you message and end the program.

Importance of the Algorithm

- Converts arithmetic logic into programmable steps.
- Ensures proper sequence and logical correctness.
- Acts as a blueprint for writing and debugging the code.
- Provides clarity and structure in program design.

4.7. Flowchart

The flowchart represents the logical flow of the calculator program. It visually outlines how the user interacts with the system, how inputs are processed, and how results are generated.

Explanation of Flow

1. Start – Program begins execution.
2. Display Menu – Lists all available arithmetic operations.
3. Take Operation Choice – User selects the desired operation.
4. Input Numbers – Program accepts two numerical values.
5. Validate Inputs – Ensures correct numeric and operator entries.

6. Perform Operation –

- Addition
- Subtraction
- Multiplication
- Division (with zero check)
- Modulus

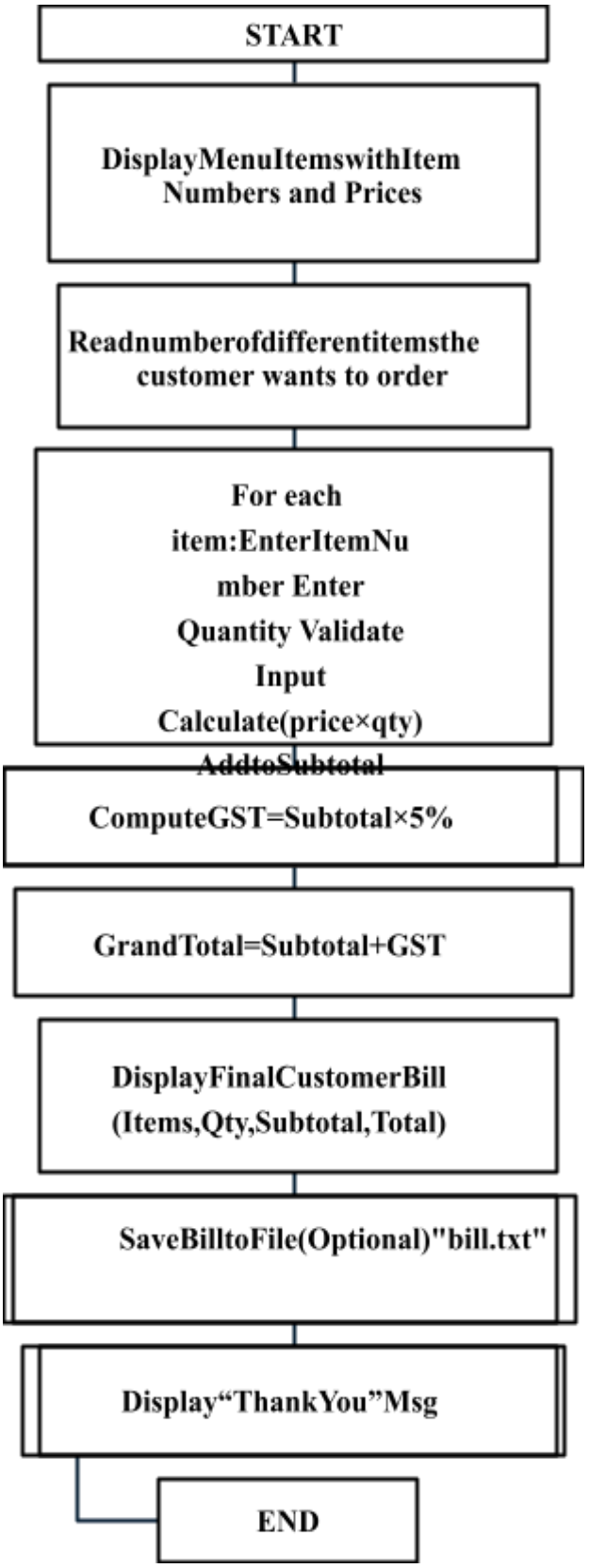
7. Display Result – Shows the final computed value.

8. Repeat? – Asks the user if another calculation is needed.

9. If Yes, loop back to the menu.

10. If No, proceed to end.

11. End – Displays a thank-you message and exits.



5.SOURCE CODE

```
#include <stdio.h>

// Function to perform calculation
double calculate(char operator, double num1, double num2) {
    switch (operator) {
        case '+':
            return num1 + num2;
        case '-':
            return num1 - num2;
        case '*':
            return num1 * num2;
        case '/':
            return (num2 != 0) ? num1 / num2 : 0; // Avoid division by zero
        default:
            printf("Error: '%c' is not a valid operator.\n", operator);
            return 0;
    }
}

int main() {
    char operator, choice;
    double num1, num2, result;

    do {
        printf("\n====== SIMPLE CONSOLE CALCULATOR ======\n");
        printf("Select an operator (+ - * /): ");
        scanf(" %c", &operator);
```

```
        printf("Enter number 1: ");
        scanf("%lf", &num1);

        printf("Enter number 2: ");
        scanf("%lf", &num2);

        // Check division by zero before calling calculate()
        if (operator == '/' && num2 == 0) {
            printf("Error: Division by zero is undefined.\n");
        } else {
            result = calculate(operator, num1, num2);
            printf("Result: %.2lf %c %.2lf = %.2lf\n", num1, operator, num2, result);
        }

        printf("\nDo you want to perform another calculation? (Y/N): ");
        scanf(" %c", &choice);

    } while (choice == 'Y' || choice == 'y');

    printf("\nThank you for using the calculator!\n");

    return 0;
}
```

SOURCE CODE EXPLANATION: CONSOLE CALCULATOR

1) Headers

```
#include <stdio.h>
```

Explanation

- stdio.h is required for all input/output functions such as
  `printf()` → prints text
  `scanf()` → reads user input

The calculator is simple, so no extra libraries are needed.

---

2) Function: calculate()

```
double calculate(char operator, double num1, double num2) {
    switch (operator) {
        case '+': return num1 + num2;
        case '-': return num1 - num2;
        case '*': return num1 * num2;
        case '/': return num2 != 0 ? num1 / num2 : 0;
        default:
            printf("%c is not a valid operator.\n", operator);
            return 0;
    }
}
```

Explanation

- This is a separate function that performs all arithmetic.
- It accepts:
  - an operator (`+ - * /`)
  - two numbers (num1, num2)
- Uses a switch-case to choose which operation to perform.
- Division includes:

  `num2 != 0 ? num1/num2 : 0;`

  to avoid division by zero and prevent program crash.

---

3) main() Function – Variables

```
char operator;
double num1, num2, result;
```

Purpose

- `operator` stores the symbol entered by the user.
- `num1` and `num2` are input numbers.
- `result` stores the output returned by the `calculate()` function.

---

4) Input: Operator and Numbers

```
printf("\nEnter an operator (+ - * /): ");
scanf(" %c", &operator);

printf("Enter number 1: ");
scanf("%lf", &num1);

printf("Enter number 2: ");
scanf("%lf", &num2);
```

Explanation

- The space before `%c` in `scanf(" %c",...)` clears unwanted newline characters.
- Reads operator and numbers from the user.
- `%lf` is used for reading double-precision floating values.

---

5) Call the calculate() Function

```
result = calculate(operator, num1, num2);
```

Explanation

- Sends the operator and the two numbers to `calculate()`.
- Receives the result based on the operation chosen.

---

6) Check for Division by Zero

```
if (operator == '/' && num2 == 0) {
```

```
    printf("Error: Division by zero is undefined.\n");
} else {
    printf("\nResult: %lf\n", result);
}
```

Explanation

- Although calculate() already prevents zero-division, this check provides a clear error message to the user.
- Otherwise, prints the result.

---

7) Program Exit

```
return 0;
```

Explanation

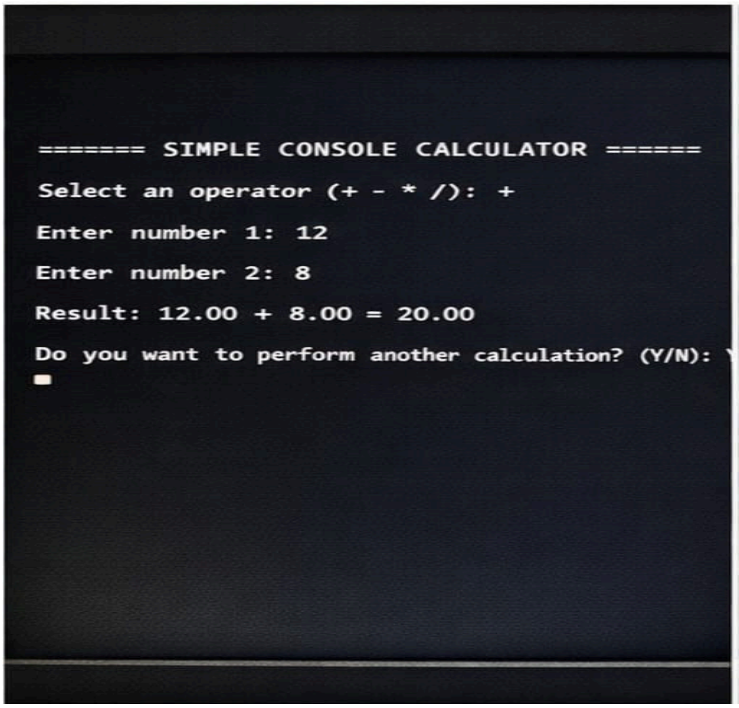
- Normal and safe program termination.

CONTROL FLOW SUMMARY

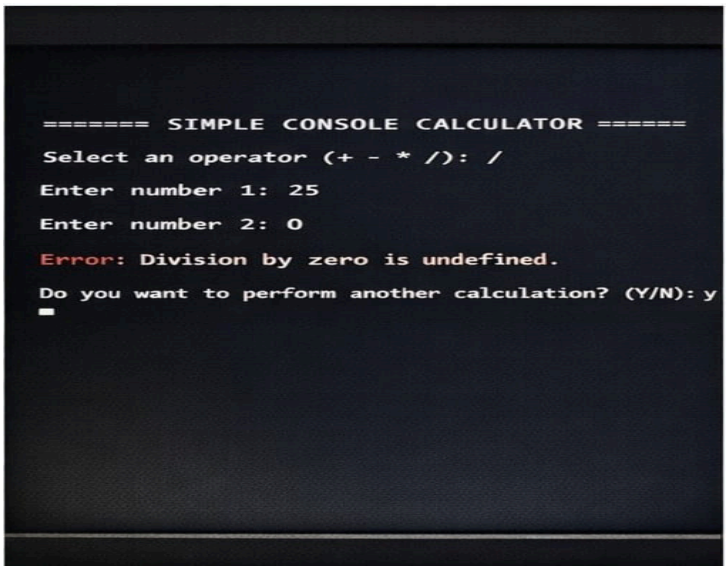
1. Ask user to enter an operator.
2. Read two input numbers.
3. Send values to `calculate()` function.
4. Perform corresponding arithmetic.
5. Check for division by zero (if operator is `/`).
6. Print output or error message.
7. End program.

4. INPUT AND OUTPUT SCREENS

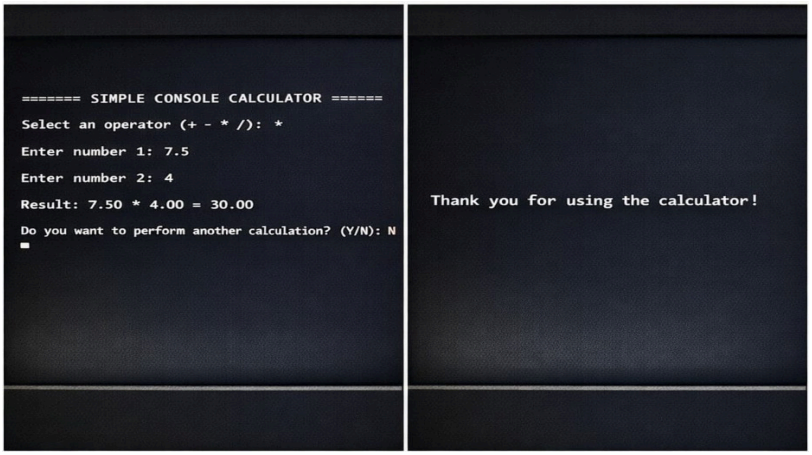
Input Screen 1:



Input Screen 2:



Output Screen:



5. DISCUSSION & CONCLUSION

5.1 Discussion

This project successfully implements a simple console-based calculator by:

- Using structured programming and modular design (separating calculation logic into a function).
- Applying control structures such as loops, conditional statements, and switch-case for operation selection and input validation.
- Demonstrating practical use of arithmetic operations in a clear, interactive, and user- friendly way.

The system is efficient, easy to use, and suitable for educational purposes, basic calculations, and small-scale arithmetic tasks.

5.2 Conclusion

The project meets its intended objectives:

- Provides a simple yet functional calculator for addition, subtraction, multiplication, division, and basic error handling.
- Demonstrates real-life application of C programming concepts, including functions, loops, and conditional logic.
- Can be easily extended in the future to include:
    - Additional operations like modulus, exponentiation, or scientific functions.
    - Continuous operation with looped menu until the user exits.
    - Graphical interface or integration with other programs.

6. REFERENCES

- Kernighan & Ritchie, *The C Programming Language*, Prentice Hall, 1988.
- TutorialsPoint – *C Programming*
- GeeksforGeeks – *Simple Console Calculator in C*
- Dev-C++ User Manual
- Programiz – *C Programming Examples*